Test Cycle 2: testing and release guide for Amit

Test the final build, fix what you find, prepare the v1.0.0 release, run the viva demo · 7–13 November 2026

AgriAgent Final Year Project · 7 November 2026

> **In one paragraph.** You test the **final build** of AgriAgent on your own Windows laptop and report every problem by **Tuesday 10 November, 23:59**. From then on **the remaining work is yours**: you fix what you found as your own pull requests (Part F), prepare the `v1.0.0` release (Part G) and run the demo in the viva (Part H). Chaitanya reviews, merges and tags. The release is published on **Wednesday 11 November** and everything is final on **Friday 13 November**. **File each FAIL as soon as you see it.** Everything you deliver is counted in the contribution record (`CONTRIBUTIONS.md`), which is regenerated from git and GitHub after the freeze. Schedule in Section 15.

Contents

1 What you need and where things go 1
2 What AgriAgent does, in five minutes 2
3 Setup on Windows 2
4 How to record results and report a FAIL 3
5 Part A: install and automated checks (about 1 hour) 4
6 Part B: regression of the 7 November fixes (about 1.5 hours) 4
7 Part C: manual cases never run before (about 1 hour) 5
8 Part D: exploratory testing (45 minutes, time-boxed) 6
9 Part E: demo material (about 1 hour) 6
10 Part F: fix what you found (Wed 11 Nov, morning) 6
11 Part G: prepare the v1.0.0 release (Wed 11 Nov, evening) 7
12 Part H: your role in the viva 8
13 Known limitations: not bugs, but note what you see 8
14 Troubleshooting 8
15 Schedule 9
16 Appendix: official labels of the six test photos 9

1 What you need and where things go

Thing	Where
Code	GitHub `Chaitanya-666/agriAgent`, branch `main`. The 7 November fixes (pull request #29) are merged; any commit from `e5f8a5b` onwards is correct. Write `git log --oneline -1` into every report.
Test photos	Drive `agriAgentShared/shared/test_photos/` (six photos). Their official labels are in the Appendix at the end of this guide.
Results table	Google Sheet **AgriAgent Test Cycle 2** at the top of `agriAgentShared`: one row per test case is already there.
Screenshots, videos	Drive `agriAgentShared/Amit/cycle2/`, named after the case, e.g. `R2_second_run.png`.
Failures	GitHub issues, one per problem (Section 4).

Questions	WhatsApp Chaitanya. If you are blocked for more than 30 minutes, ask; don't wait.

2 What AgriAgent does, in five minutes

AgriAgent takes a field photo and the crop name and marks which green patches should be sprayed with herbicide. Pretrained models find and outline the plants; plain rules and a safety gate decide what is sprayed. It is a research prototype and never drives a real sprayer.

1. **Detect.** Grounding DINO looks for boxes that match the prompt `weed . <crop> plant .`.
2. **Segment.** SAM 2 turns each box into a pixel mask.
3. **Plant rules.** Every green patch is called crop, weed or unsure: inside a crop mask or on the crop row = crop; much smaller than the crop, or off the row = weed; anything else = unsure.
4. **Safety gate.** The only part allowed to approve spraying. Unsure, low-confidence or crop-labelled things are never sprayed.
5. **Advice (cloud, optional).** Qwen 3.8 names the weed; a herbicide is suggested only from the ICAR bulletin row for the chosen crop. An optional escalation agent reviews withheld plants. These only advise.

What the screens mean

On screen	Meaning
Yellow **MOCK OUTPUT** banner	Fake test data at fixed positions; your photo is not looked at. Never a real result.
Green **REAL OUTPUT** banner	Grounding DINO and SAM 2 really ran on your photo.
Orange "No plant in this photo was recognised as …"	The crop setting is probably wrong for this photo; the result is not trustworthy.
KPI cards	Share of the photo sprayed, herbicide saved compared with spraying everything, number of zones.
Coloured patches, magenta outline	Zones approved for spraying; tiny ones get a ring so you can see them.
Checkbox "Show every plant and what the system decided" (new)	A third image: red = sprayed, green = protected as crop, yellow = left alone; soil darkened.
System 2 report	The model's diagnosis and, only if the ICAR table has a matching row, a herbicide with its bulletin page; otherwise "no evidence".
Summary for the farmer	Plain-language summary in English, Hindi or Marathi. Hindi and Marathi need Cloudflare keys.
Download Field Prescription (GeoJSON)	The approved zones as a map file. Coordinates are placeholders near VJTI, not GPS.

Mock Mode is **on** by default. "Real mode" in this guide always means the **Mock Mode** switch in the sidebar is **off**. The first real run after starting the app takes longer (the models load once, 10–40 s depending on your laptop); later runs are faster.

3 Setup on Windows

Use a **new, empty folder**, so you test the install a new user would do. Python 3.10, 3.11 or 3.12 (3.11 is what we verified; check with `python --version`). Run everything in PowerShell. **Time each step** with your phone's stopwatch and note the times in row A1 of the Sheet.

1. Get the code: `git clone https://github.com/Chaitanya-666/agriAgent agriAgent-tc2`, then `cd agriAgent-tc2`.
2. Note the commit: `git log --oneline -1`.
3. Virtual environment: `python -m venv .venv`, then `.venv\Scripts\activate`. The prompt now starts with `(.venv)`. If PowerShell refuses to run the script, run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.
4. Packages, exact tested versions: `python -m pip install -r requirements-lock.txt`. This downloads PyTorch with CUDA (about 2.5 GB). It also works on a laptop without an NVIDIA GPU, only slower.
5. The project itself: `python -m pip install -e . --no-deps`
6. Settings file: `copy .env.example .env`
7. Model weights, about 1.7 GB, once: `python scripts\download_models.py`. Then `python scripts\download_models.py --check` should say the models are present.
8. Test photos: download the six photos from Drive into a folder you can find, e.g. `Downloads\test_photos`.
9. Start the app: `streamlit run app\app.py`. A browser tab opens at `http://localhost:8501`. To stop it, press Ctrl + C in PowerShell.

Cloudflare key (optional, recommended). Hindi/Marathi summaries and the System 2 diagnosis need a reasoning model in the cloud. Create a free Cloudflare account; open Dashboard → AI → Workers AI → "Use REST API"; copy the **Account ID**; create a token with the **Workers AI** template. Put both into `.env`:

```
CLOUDFLARE_ACCOUNT_ID=<your account id>
CLOUDFLARE_API_TOKEN=<your token>
```

Restart the app after editing `.env`. Never send keys on WhatsApp and never commit `.env` (git ignores it already). Without keys everything still runs: cases marked **(keys)** then only check that the app clearly says the model was unavailable and does not crash. Write "no keys" in the Sheet.

4 How to record results and report a FAIL

The Sheet. Fill one row per case: Date, Commit tested, Verdict, What you saw, Screenshot / video file, GitHub issue #.

Verdict	Use when
PASS	Everything in the "Pass if" column happened.
FAIL	Anything in "Pass if" did not happen, or the app crashed or showed a red error. File an issue.
NOTE	It worked, but something was confusing, slow, ugly or worded badly. Write it down; it may still become a fix.

"What you saw" is the most useful column: numbers on screen (percentages, zone count, times), exact error text, what looked wrong. "Worked fine" alone does not help.

A GitHub issue for each FAIL. At github.com/Chaitanya-666/agriAgent/issues → New issue. Title: `[TC2] <case ID>: <what went wrong>`, e.g. `[TC2] R2: meta tensor error after changing crop`. Body:

```
Commit:      (git log --oneline -1)
Setup:       Windows version, Python version, GPU or CPU, mock or real, keys yes/no
Steps:       1. ...  2. ...  3. ...
Expected:    (copy the "Pass if" text from this guide)
Actual:      (what you saw, exact error text; paste the PowerShell output if there is
any)
Screenshot:  (drag the image into the issue, or the Drive file name)
```

Then put the issue number in the Sheet row. One issue per problem. Unsure whether it is a bug? File it anyway and say so: a wrong alarm costs a minute, a missed bug costs the demo.

Good evidence. A screenshot of the **whole** browser window (Win + Shift + S), plus the last lines of the PowerShell window where `streamlit` runs: errors often appear there and not in the browser.

5 Part A: install and automated checks (about 1 hour)

Run these in the activated `.venv`, from the project folder.

ID	Do	Pass if
A1	The whole setup of Section 3, timed.	Every step finishes without an error. If one fails, copy the full error into the issue.
A2	`python -m unittest discover -s tests`	The last line is `OK` (about 270 tests, under a minute). A few lines of warnings above it are fine.
A3	`python tests\e2e\runner.py --all`	`Passed: 35`, `Failed: 0`.
A4	`python scripts\verify_cycle0_issues.py`	Every result line starts with `[PASS]`. It drives the app headlessly and may take a few minutes.
A5	`python scripts\verify_test_cycle1.py` (mock cases only, seconds)	ST-01 … ST-11 all `[PASS]`.
A6	**(keys)** `python scripts\check_providers.py`	Two PASS lines (cloudflare-qwen, cloudflare). Without keys: a clear “missing key” message, no crash.

6 Part B: regression of the 7 November fixes (about 1.5 hours)

Each case checks one fix made on 7 November. Start the app (`streamlit run app\app.py`). Before each case, set the sidebar back to: Crop as stated, Language English, agent off, Force System 2 off.

ID	Do	Pass if
R1 (F24)	1. Mock Mode on, tick the sample photo, wait for the yellow MOCK banner. 2. Switch Mock Mode **off**. 3. Watch the page while the real run starts.	The old MOCK result disappears at once; only the spinner is visible until the new REAL result appears. Before the fix, the old MOCK page stayed visible, greyed out.
R2 (F26)	1. Restart the app (Ctrl + C, start again). 2. Real mode, Crop **Sugar Beet**, upload `beet_X3-130-0.png`; time the run. 3. **While the spinner is still running**, change Crop to Cotton and back to Sugar Beet, three times quickly. 4. Upload `beet_X-10-0.png`; time this run too.	No error at any point; in particular no “Cannot copy out of meta tensor”. The second photo is clearly faster than the first run (the models now load once per app start). Write both times in the Sheet.
R3 (F27)	1. Real mode, upload `beet_X3-130-0.png`. 2. Set Crop to **Cotton** (wrong on purpose). 3. Scroll to “Summary for the farmer”. 4. Read it in English, then **(keys)** Hindi and Marathi.	An orange warning at the top says no plant was recognised as Cotton. The summary also contains the warning — English: “Warning: no plant in this photo was recognised as cotton…”; Hindi (usually

		at the end): “चेतावनी: इस फ़ोटो में कोई भी पौधा कपास के रूप में नहीं पहचाना गया…”; Marathi: “इशारा: या फोटोमधील कोणतेही रोप कापूस म्हणून ओळखले गेले नाही…”.
R4 (F29)	**(keys)** 1. Real mode, Crop **Cotton**, tick **Force System 2**. 2. Upload `cotton_CM_17.jpg`. 3. Read the System 2 report. 4. Read the farmer summary in English, then Hindi.	If a herbicide is shown, the summary **ends** with the caveat — English: “…not an instruction: read the product label and ask your local agriculture officer before spraying.”; Hindi: “यह ICAR तालिका का सुझाव है, निर्देश नहीं: …”. The summary never orders the farmer to spray a dose. If no herbicide is shown, write “no prescription” (allowed).
R5 (F28)	1. Real mode, Sugar Beet, upload `beet_X2-30-0.png`. 2. Under the two images tick **Show every plant and what the system decided**. 3. Compare with the labels in the Appendix.	A third image appears: soil darker, plants red, green or yellow, and a caption with three percentages. Expect **two of the three beets red** (known failure F1, Section 13) and small red specks on weeds. Screenshot it.
R6 (F28)	Same as R5 with `cotton_CM_17.jpg`, Crop Cotton.	The large palmer amaranth weeds are **green** (the detector called them cotton: known limitation); small specks red. Screenshot it; it is a report figure.

7 Part C: manual cases never run before (about 1 hour)

ID	Do	Pass if
MT-04	**(keys)** Sugar Beet, `beet_X3-130-0.png`. Switch the summary language English → Hindi → Marathi → English.	Each switch is instant (no new spinner). Judge the language as a reader: correct words (चुकंदर, खरपतवार; साखर बीट, तण, फवारणी), natural sentences, the same numbers as English. Copy any spelling mistake exactly into the Sheet.
MT-05	Sugar Beet, `beet_X3-130-0.png`. Click **Download Field Prescription (GeoJSON)**, save as `MT-05_X3-130-0.geojson`, then drag the file onto geojson.io.	The page does not rerun when you click download. On the map the shapes appear near VJTI (Matunga, Mumbai), a few metres across, the right way up (compare with the photo). Click a shape: its properties include `buffer_margin_px` and `approved_by`. Upload the file to `Amit/cycle2/`.
MT-06	Sugar Beet, upload `beet_X-10-0.png`; open its labels (Appendix).	The coloured patches sit on the 2 red-boxed weeds, not on the 4 beets.
MT-07	Sugar Beet, upload `beet_X2-30-0.png`.	Known failure: 2 beets touched. Describe what you see; it goes in the report as the main limitation.

8 Part D: exploratory testing (45 minutes, time-boxed)

Try to break it. Each idea is one Sheet row (`X1` … `X8`); add your own. For each, write what you expected, what happened, and a screenshot if anything looks wrong.

ID	Try	Expect
X1	Your own phone photo of plants: garden, potted plant, roadside weeds. Try both crops.	No crash; a result that looks sensible; describe it.
X2	A photo with no plants: a desk, a wall, a dark room.	Nothing sprayed (0%).
X3	A very large photo straight from your phone camera, and a tiny one (a small screenshot).	Both run; large ones may be slow; no crash.
X4	Portrait and landscape photos; a PNG with transparency.	All run.
X5	The app open in two browser tabs at once, running different photos.	Both finish with their own results.
X6	Change every sidebar control during a run; use the browser's back and refresh buttons.	No crash; the final screen matches the final settings.
X7	Real mode with Wi-Fi turned off (after the models are downloaded).	The vision part still works; a clear message that the reasoning model was unreachable; no crash.
X8	Your own idea.	—

9 Part E: demo material (about 1 hour)

E1, screen recording (our backup if the live demo fails). Win + G (Xbox Game Bar) or OBS, 3–5 minutes, with your voice. Script:

1. Say: "AgriAgent, real mode, sugar beet." Show the sidebar with Mock Mode off.
2. Upload `beet_X-10-0.png`. Point at the REAL banner and the KPI cards.
3. Show the spray map next to the photo: the two weeds are marked, the four beets are not.
4. Tick "Show every plant…": explain red, green, yellow.
5. Scroll to the farmer summary; switch to Hindi **(keys)**.
6. Download the GeoJSON and drop it on geojson.io.

Save it as `E1_demo_recording.mp4` in `Amit/cycle2/`.

E2, demo playbook. Open `docs/guides/04_Demo_Playbook.pdf` and do it step by step on your laptop. In the Sheet, mark each step ok or problem, and list the steps that no longer match the app.

10 Part F: fix what you found (Wed 11 Nov, morning)

Every FAIL you filed becomes yours to fix, one pull request per issue. Chaitanya reviews and merges; nothing reaches `main` without that review. If you cannot find the fix, a comment on the issue that names the cause (file, line, why) is still a contribution.

Workflow for each issue

1. Start from the latest code: `git switch main`, `git pull`, then a branch named after the issue: `git switch -c amit/fix-<issue number>-<short-name>` (e.g. `amit/fix-41-hindi-caveat`).
2. Reproduce the problem with the steps from your issue.
3. Find the code (table below). Write a test in `tests/` that fails because of the bug, then fix the code until it passes.

4. Run the checks: `python -m unittest discover -s tests` (OK), `ruff check .` and `ruff format .`.
5. Commit with a message like `fix(app): Hindi summary keeps the label caveat (TC2 #41)` and push: `git push -u origin amit/fix-41-hindi-caveat`.
6. Open a pull request into `main`; the description template asks what, why and how you tested. Write `Fixes #41` in it.
7. Answer review comments with more commits on the same branch. Chaitanya merges when it is right.

Where to look

Symptom	File
Screen layout, banners, KPI cards, buttons, language box	`app/app.py`, `app/components/ui.py`
Upload, GeoJSON file, KPIs	`app/components/pipeline.py`
Farmer summary text, Hindi/ Marathi, safety sentences	`src/agronomy/farmer_summary.py`
Herbicide advice, "no evidence"	`src/agronomy/prescription.py`, `src/agronomy/knowledge_store.py`
Cloud model errors, keys	`src/agronomy/providers/`, `scripts/check_providers.py`
Model loading, crashes before a result	`src/vision/grounding_dino.py`, `src/vision/sam2_wrapper.py`, `src/vision/model_cache.py`
Coloured overlays, the "every plant" view	`src/vision/visualization.py`

Do not change without asking Chaitanya first (these are frozen and measured in the report): the plant rules in `src/vision/plant_candidates.py`; the gate thresholds (0.70 confidence, 0.85 mask quality, 85% infestation); anything in `data/manifests/` or `docs/results/`. Never commit `.env`, photos or model weights; add no new package without asking.

11 Part G: prepare the v1.0.0 release (Wed 11 Nov, evening)

When all fix PRs are merged and Chaitanya says "go":

ID	Do	Pass if
G1	On the latest `main`, run Part A again (A2–A5).	All pass; write the commit in the Sheet.
G2	Branch `amit/release-1.0.0`: set `version = "1.0.0"` in `pyproject.toml`; run `python scripts/contributions.py` (it rewrites `CONTRIBUTIONS.md` with the latest counts); write `RELEASE_NOTES.md` (what the release contains, the fixes from Test Cycle 2 with their issue numbers, known limitations from Section 13). Open the PR.	CI green; Chaitanya merges and creates the tag `v1.0.0`.
G3	`python scripts\package_release.py --ref v1.0.0`	Prints OK; writes `dist\AgriAgent-…-source.zip` and a `.sha256` file. Upload both to Drive `Amit/release/`.
G4	Unzip into a **new** folder and do the setup of Section 3 from the zip (not from git), then A2.	`OK`. This proves the submitted zip works on its own.

G5	On GitHub → Releases → "Draft a new release" for tag `v1.0.0`: title "AgriAgent v1.0.0", paste the release notes, attach the zip and `.sha256`. Save as **draft**.	Chaitanya publishes it after a look.

12 Part H: your role in the viva

- **Live demo.** You run it on your laptop, following the E1 script. Keep the E1 recording open in a second window as the backup.
- **Testing slide.** You present the "Testing" slide: what Test Cycles 0 and 2 found and how each was fixed. Use your own issues as examples.
- **Rehearsal.** Two full run-throughs with Chaitanya on Thursday 12 and Friday 13 November, timed.

13 Known limitations: not bugs, but note what you see

These are known and measured; they go in the report. Don't file them as new bugs unless something is **worse** than described here.

What you may see	Why
Whole beets coloured in `beet_X2-30-0.png` (finding F1)	Rule 4 calls any plant off the crop row a weed when the detector missed its crop box.
Few or no weeds sprayed on cotton photos	The rules assume weeds are smaller than the crop; in the cotton close-ups a big weed can be taken for the crop. Cotton weed recall is about 10%.
Small specks sprayed next to beets on `beet_X3-130-0.png`	Tiny sprouts inside beet boxes; about 0.05% of the photo.
Escalation agent says "unsure" for almost everything	Measured: it is safe but rarely decides "weed". It is off by default.
Marathi summary with odd spellings	The cloud model makes spelling mistakes; still note each one exactly (MT-04).
GeoJSON shapes near VJTI, not your field	Coordinates are placeholders, not GPS; the file says so.
PowerShell shows "You are using a model of type sam2_video to instantiate…" and FutureWarnings	Harmless log lines.
First real run slow (10–40 s), much slower without an NVIDIA GPU	Models load once per app start; on CPU every run is slower. Note your times, don't file them as bugs unless a run takes over 2 minutes.

14 Troubleshooting

Problem	Try
`.venv\Scripts\activate` is blocked	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, then activate again.
`pip install` fails on torch	Check Python is 3.10–3.12 and 64-bit; free about 6 GB of disk; report the full error (A1 FAIL).

`streamlit` is not recognised	The `.venv` is not active; activate it again.
Browser does not open	Open `http://localhost:8501` yourself.
Port already in use	Another app is running: close the old PowerShell window, or `streamlit run app\app.py --server.port 8502`.
Red "Real mode requested but … could not be loaded"	Run `python scripts\download_models.py` again; if it repeats, that is a FAIL.
Hindi/Marathi stays English	No keys, a wrong key, or no network. The orange note under the summary says which. With keys set, restart the app.
Anything else	Screenshot + PowerShell output → WhatsApp Chaitanya, then file it.

15 Schedule

When	What
Sun 8 Nov	Part A. Report install problems the same day (they block everything else).
Mon 9 Nov	Parts B and C.
Tue 10 Nov	Parts D and E; Sheet complete by 23:59; WhatsApp Chaitanya "TC2 done" with the number of FAILs. For Parts F–H, add your own rows at the bottom of the Sheet as you go: `F-<issue number>` per fix PR, `G1` – `G5`, `H1` – `H3` (demo, testing slide, rehearsals).
Wed 11 Nov	Morning: Part F, a fix PR for each of your FAIL issues, review rounds. Evening: Part G (release PR, tag, package, verify, draft release).
Thu 12 Nov	Viva rehearsal 1 (Part H); final numbers in report and slides (Chaitanya).
Fri 13 Nov	Viva rehearsal 2. Everything final.

16 Appendix: official labels of the six test photos

Green box = crop plant, red box = weed. Copied unchanged from the dataset files. Cotton photos: the dataset labels only weeds; cotton is never boxed.

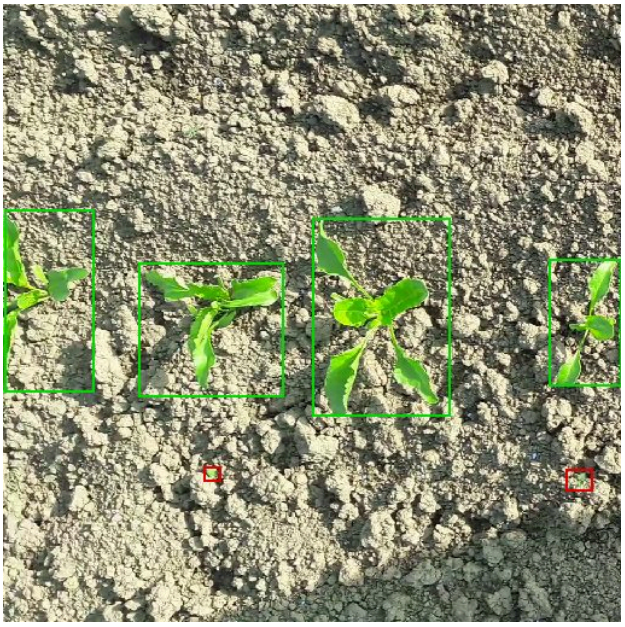


Figure 1: `beet_X-10-0.png` : 4 beets, 2 weeds

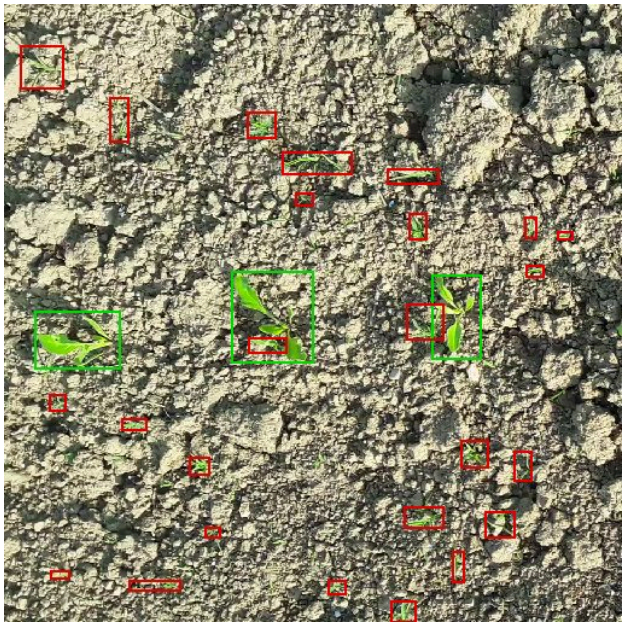


Figure 2: `beet_X2-30-0.png` : 3 beets, 25 weeds

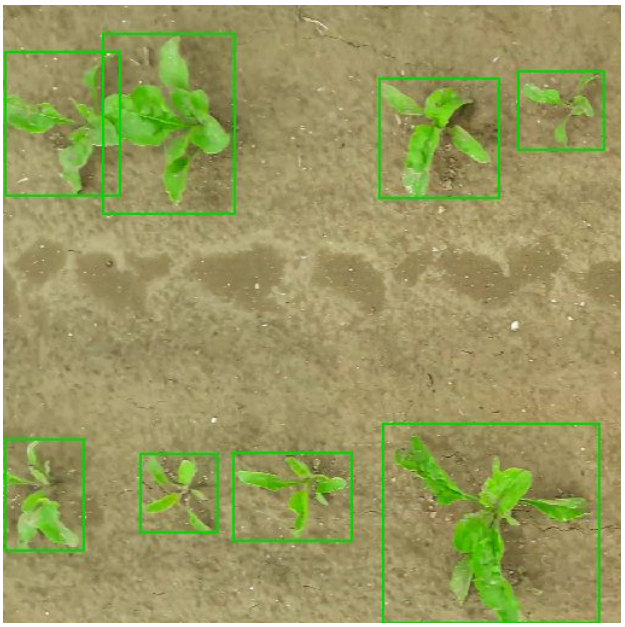


Figure 3: `beet_X3-130-0.png` : 8 beets, 0 weeds



Figure 4: `beet_X3-430-0.png` : 9 beets, 1 weed

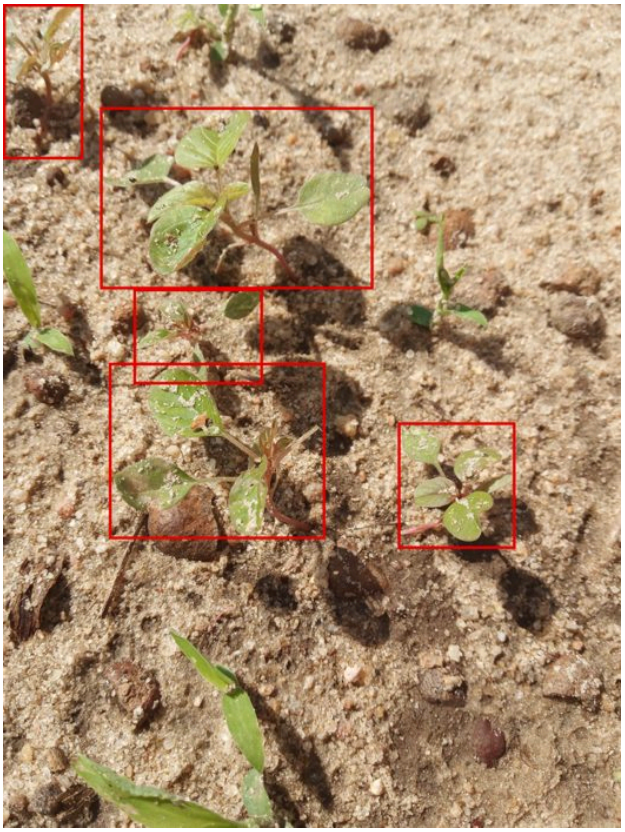


Figure 5: `cotton_CM_17.jpg` : 5 palmer amaranth

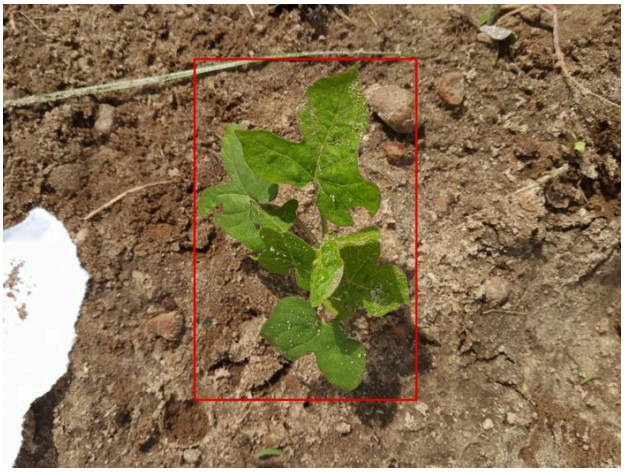


Figure 6: `cotton_CM_11.jpg` : 1 morningglory